MCA SEMESTER IV · RESEARCH PROJECT · AUGUST 2026

WealthAI

AI-Powered Personal Finance Assistant with Machine Learning Classification, Spending Outlier Analytics and Stochastic Projections

Submitted by

Deviya HarshitKumar KalpeshKumar

USN 241VMTR01558 · Elective: Data Analytics

UNDER THE GUIDANCE OF

Prof. Prabha S

Faculty Guide, Department of Computer Applications

Jain Online

(Deemed-to-be University), Bengaluru

In partial fulfilment of the requirements for the award of

Master of Computer Applications

Academic Year 2025-26

Personal finance is fragmented, manual and static

Users hold money across banks, brokers, wallets and cards. No single view of net worth exists, and the tools that do exist stop at record-keeping.

01

Information fragmentation

Accounts spread across multiple banks, brokers, card issuers and wallets. Computing total net worth is manual labour.

02

Slow, error-prone categorisation

Hundreds of raw monthly transaction descriptions must be labelled by hand. Budget tracking collapses under user fatigue.

03

Late anomalies, static forecasting

Double charges and silent price hikes surface only at month end. Goal calculators still use fixed-rate A = P(1+r)^t, ignoring volatility.

What WealthAI sets out to do

AIM

Design, implement and deploy an automated, full-stack, AI-driven personal finance application that consolidates user records, categorises transactions with machine learning, flags irregular spending, evaluates long-term goals using stochastic Monte Carlo simulation, tracks live asset prices and answers financial questions through a conversational interface.

FUNCTIONAL OBJECTIVES · 7 CORE MODULES

- Automated CSV import and ML categorisation
- Budgets, savings goals and subscription tracking
- Portfolio management with live market quotes
- Spending anomaly detection
- Short-term expense forecasting
- Stochastic goal and portfolio projections
- Interactive financial chatbot

Where existing approaches fall short

Domain	Conventional approach	Limitation	Adopted in WealthAI
Transaction categorisation	Manual entry and fixed rule tables	Does not generalise to unseen merchant strings	TF-IDF vectorisation + Multinomial Naive Bayes
Anomaly detection	Fixed amount thresholds	High false-positive rate; ignores personal baselines	Isolation Forest, Z-score fallback under 15 records
Expense forecasting	Simple month-on-month comparison	No trend fitting across cycles	OLS linear regression on monthly aggregates
Goal projection	Deterministic compound interest A = P(1+r)^t	Assumes fixed return; hides market risk	1,000-iteration Monte Carlo, P10 / P50 / P90 bands

RESEARCH GAP

No single open personal-finance platform combines NLP categorisation, unsupervised anomaly detection and stochastic projection behind one authenticated interface.

Agile, eight weekly sprints, single developer

Waterfall rejected — cannot absorb the requirement churn that appears when ML behaviour is validated against real data. Spiral rejected — risk-analysis overhead is disproportionate to an eight-week single-developer project.



KEY RISKS AND MITIGATIONS (Table 3.2)

Risk	Probability / Impact	Mitigation
Cold-start delay on Render free tier	Medium / Low	Lightweight GET /health ping checks during startup
Categoriser unreliable on sparse data	Medium / Medium	Rule-based keyword dictionary fallback
Unstable public market data responses	Low / High	Exception wrappers around yfinance; cached close quotes

CHAPTER 4 · TECHNOLOGY STACK

MERN foundation, Python microservice for the analytics

PRESENTATION

React.js + Vite
Single-page application

Material UI
Component system

Recharts
Charts and visualisation

Axios
REST client

APPLICATION

Node.js + Express.js
REST API layer

JWT middleware
Route authorisation

bcryptjs (salt 10)
Password hashing

Mongoose ODM
Data modelling

MACHINE LEARNING

Python + Flask
Analytics microservice

scikit-learn
TF-IDF, Naive Bayes, IsolationForest

pandas / NumPy
Aggregation and simulation

yfinance
Live market data gateway

DATA & CLOUD

MongoDB Atlas
Managed cluster, indexed per user

Vercel
Frontend hosting

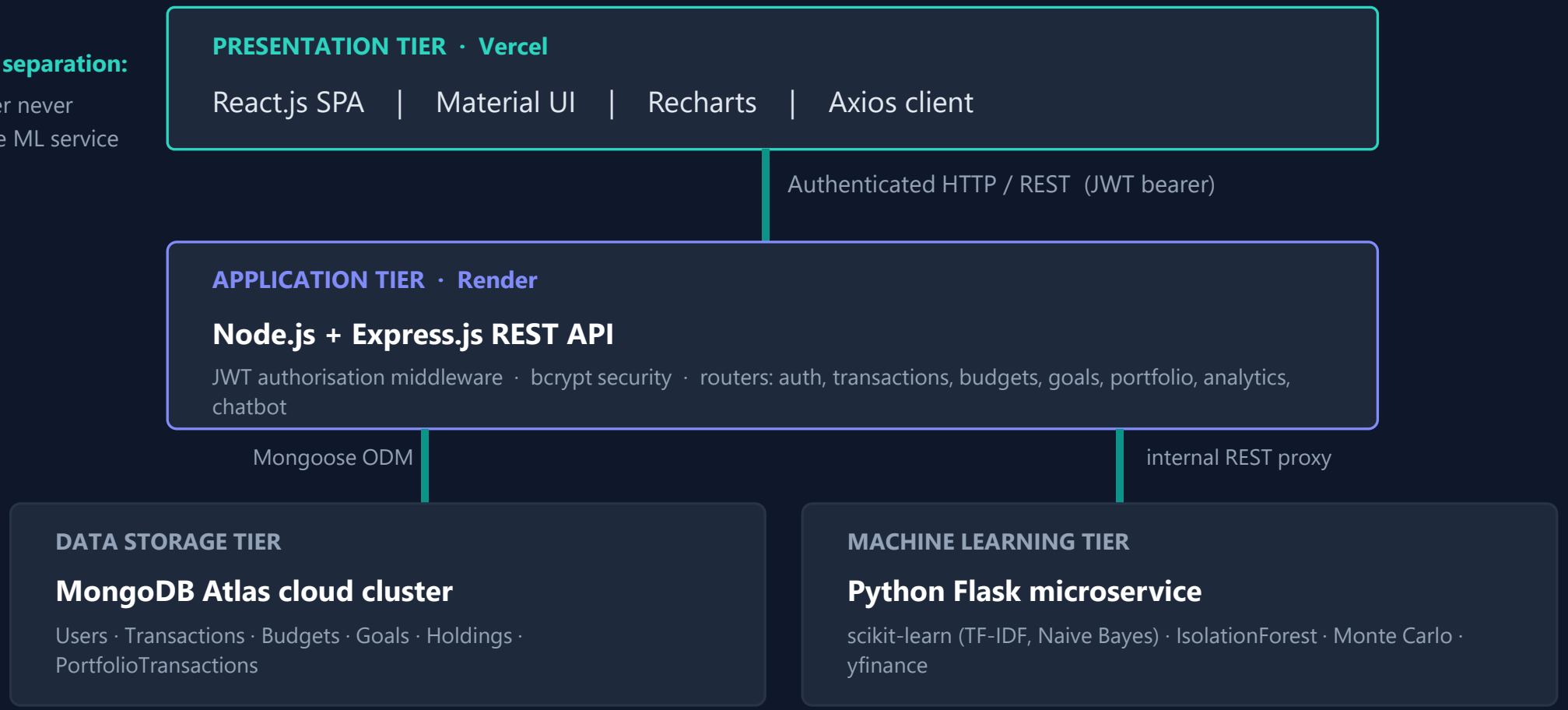
Render
API + ML service hosting

GitHub
Version control

CHAPTER 4 · FIGURE 4.1

Three-tier microservice architecture

Strict tier separation: the browser never reaches the ML service directly.



CHAPTER 4 · TABLE 1.1

Machine learning pipelines and hyperparameters

System function	Model	Configuration	Fallback rule
Transaction categorisation	Multinomial Naive Bayes	`TfidfVectorizer(lowercase=True, stop_words='english', ngram_range=(1,2))`	Keyword dictionary match
Anomaly detection	Isolation Forest	`n_estimators=100, contamination=0.05, random_state=42`	Z-score > 2.5 when n < 15
Expense forecasting	OLS Linear Regression	`LinearRegression() on monthly aggregate expenditure`	Rolling average when span < 3 months
Goal / portfolio projection	Monte Carlo simulation	`N = 1000 iterations, Gaussian returns (mu, sigma), P10 / P50 / P90`	Deterministic compound interest
Market data retrieval	yfinance gateway	`Real-time ticker quotes and symbol search`	Cached historic close quotes

Every pipeline has a deterministic fallback, so the product never fails closed when the model lacks data.

CHAPTER 4 · TABLES 4.1 – 4.3

Data model and service interfaces

MONGODB COLLECTIONS

Collection	Key attributes
User	name, email (unique), password (bcrypt hash)
Transaction	date, description, amount, category, ref: User
Budget	category, limitAmount, period
Goal	title, targetAmount, currentAmount, targetDate
Holding	symbol, shares, buyPrice, assetType
PortfolioTransaction	symbol, type (BUY/SELL), shares, price

Every collection is indexed on the user reference, so each query is scoped to one authenticated account — multi-tenant isolation at the data layer.

FLASK ML MICROSERVICE ENDPOINTS

Endpoint	Returns
`POST /predict/category`	TF-IDF + Naive Bayes category
`POST /predict/anomaly`	Isolation Forest / Z-score outliers
`POST /predict/expense`	Linear regression spend forecast
`POST /portfolio/quote`	Live yfinance price quotes
`POST /portfolio/analyze`	Allocation and risk metrics
`POST /portfolio/simulate`	1,000-iteration Monte Carlo bands
`POST /chatbot`	Context-aware financial guidance

Express exposes seven routers: auth, transactions, budgets, goals, portfolio, analytics, chatbot.

SECTION 02

Live System Demonstration

Running against the deployed cloud environment — not a local build.

- Register / JWT login
- Dashboard
- Add transaction
- Budgets & goals
- Portfolio
- Aurelia chatbot
- Reports

Deployed and live on cloud infrastructure

Frontend client
React SPA on Vercel

`finance-assistant-pearl-nu.vercel.app`

Express API
Node.js REST layer on Render

`finance-server-zfco.onrender.com`

Flask ML service
Python analytics on Render

`finance-ml-service-pw5a.onrender.com`

Database
MongoDB Atlas managed cluster

`cluster0 · indexed per user`

ALL SEVEN FUNCTIONAL MODULES OF SECTION 1.3.2 WERE DELIVERED

Evaluation metric (Table 5.1)	Measured value
TF-IDF + Naive Bayes classification accuracy	___ %
Isolation Forest anomaly detection precision	___ %
Monte Carlo latency per 1,000 iterations / average REST latency	___ ms

What the analysis showed

1 Integrated NLP classification

TF-IDF vectorisation paired with Naive Bayes gave rapid, accurate classification of standard transaction descriptions at very low computational cost.

2 Adaptive anomaly detection

Isolation Forest detected unusual transactions reliably on larger datasets, while the Z-score fallback held up for accounts with fewer than 15 transactions.

3 Stochastic beats static forecasting

Monte Carlo projections factored in market volatility and produced realistic outcome ranges, outperforming fixed-return calculators.

4 Microservice isolation as security

Routing every client request through the Node.js API kept the Python ML service off the public browser path and preserved tenant isolation.

CHAPTER 5 · HONEST ASSESSMENT

Known limitations

Free-tier cold starts	The Render free tier idles the ML service; the first chatbot or analytics call after an idle period takes noticeably longer. Mitigated with health pings, not eliminated.
Evaluation on a modest dataset	Model metrics were measured on the project's own test ledger and a sample bank-statement CSV, not on a large public benchmark corpus.
CSV-only ingestion	No direct bank connectivity. Statements must be exported and uploaded manually.
Mobile parity gap	The mobile navigation does not yet expose the quick-add, search and avatar menu that the desktop navbar provides.
No conversation history	Chatbot sessions are not persisted across logins — this needs a new collection and CRUD routes.

CHAPTER 5 · RECOMMENDATIONS AND FUTURE SCOPE

Where WealthAI goes next

NEAR-TERM RECOMMENDATIONS

Model retraining pipeline

Periodically retrain Naive Bayes on anonymised user-verified labels.

Redis caching layer

Cache market quotes to cut repeated yfinance calls.

Open banking integration

Plaid or Account Aggregator feeds alongside CSV upload.

Custom classification rules

Let users override ML categories with their own keywords.

LONGER-TERM SCOPE

Native mobile apps

React Native clients re-using the existing REST API.

LLM-backed advisor

Expand the chatbot into a fully conversational financial advisor.

Tax optimisation module

Automated tax-bracket analysis and planning.

Multi-currency support

Automatic conversion for international accounts.

CHAPTER 5 · CONCLUSION

WealthAI closes the loop from ledger to forecast

WealthAI addresses the four core problems of personal finance management — data fragmentation, manual categorisation effort, undetected anomalies and static goal forecasting — by combining a MERN web interface with a dedicated Python machine learning microservice, deployed and running on cloud infrastructure.

- NLP categorisation
- Isolation Forest outliers
- Linear regression forecasts
- Live market tracking
- 1,000-run Monte Carlo projection

Thank you

Questions welcome.

Deviya HarshitKumar KalpeshKumar
USN 241VMTR01558 · MCA Semester IV
Guide: Prof. Prabha S
Jain Online (Deemed-to-be University)